Lab 11: Merge Sort and Quick Sort

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Divide-and-conquer sorting, partitioning, complexity analysis

Objectives

- Implement merge sort and understand its $O(n \log n)$ divide-and-conquer structure.
- Implement quick sort with the Lomuto partition scheme; analyse best, average, and worst cases.
- Measure empirical running times and compare with theoretical predictions.
- Apply sorting to practical problems: finding duplicates and the k-th smallest element.

Quick Reference

Merge sort.

```
void mergeSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergeSort(a, lo, mid);
    mergeSort(a, mid+1, hi);
    merge(a, lo, mid, hi);
}
```

Quick sort pivot (Lomuto). Partition around `a[hi]`; elements $<$ pivot go left.
Worst case. Already-sorted input with last-element pivot $\Rightarrow O(n^2)$.
Fix. Swap a random element with `a[hi]` before partitioning.

Algorithm	Worst	Average
Merge sort	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n^2)$	$O(n \log n)$

Quick Experiments (Try It)

Try It 1: Merge sort on paper first

Trace merge sort on $[5, 2, 8, 1, 9, 3]$ by hand. Draw the recursive split tree and the merge steps. How many total element comparisons are made?

Try It 2: Quick sort worst case

Instrument your partition function with a comparison counter. Compare the total comparisons for a sorted array of 10 vs. a random array of 10.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Implement Merge Sort *(40 min)*

Implement `void mergeSort(int[] a, int lo, int hi)` and the helper `void merge(int[] a, int lo, int mid, int hi)`.

- Use a temporary array of size `hi - lo + 1` inside `merge`.
- Print the array state after each top-level merge call.

- Test on: `{5,2,8,1,9,3}`, a reversed array, an already-sorted array.

In-Lab Exercise 2: Implement Quick Sort *(40 min)*

Implement `void quickSort(int[] a, int lo, int hi)` using the Lomuto partition (pivot = `a[hi]`), where `int partition(int[] a, int lo, int hi)` returns the final pivot index.

- Print the array after each `partition` call.
- Add a randomised-pivot variant.
- Compare call counts with merge sort on the same arrays.

In-Lab Exercise 3: Empirical Comparison *(20 min)*

Generate arrays of size 10 000: random, sorted, reverse-sorted. Time merge sort and quick sort (random pivot) using `System.nanoTime()`. Record in a table. Does the ratio $T_{\text{merge}}/T_{\text{quick}}$ stay roughly constant?

In-Lab Exercise 4: Find Duplicates *(20 min)*

Given an unsorted `int[]`, find all duplicate values by sorting then scanning linearly. Overall complexity? Compare with brute-force $O(n^2)$. Test with `{4, 1, 3, 2, 4, 7, 3}` — expected duplicates: `3, 4`.

AI Collaboration

Suggested prompts:

- "Trace the Lomuto partition on $[3, 6, 8, 10, 1, 2, 1]$ with pivot $= 1$."
- "Why does randomising the pivot make $O(n^2)$ behaviour negligibly unlikely?"

Verify: Ask the AI for the sorted result and pivot indices for a specific input. Run your code on the same input and compare.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Merge sort trace

Trace `mergeSort(a, 0, 4)` on $a = [7, 3, 5, 1, 4]$. Show the recursive call tree and array contents after each merge.

Trace & Predict 2: Partition trace

Trace `partition(a, 0, 5)` (Lomuto) on $a = [8, 3, 1, 5, 6, 2]$, pivot = `a[5] = 2`. Show `i`, `j`, and the array after each iteration. What is the final pivot index?

Write Code on Paper 1: QuickSelect: k-th smallest

Using quick sort's partition, write `int kthSmallest(int[] a, int k)`. After partitioning at index p: if $p = k - 1$ return `a[p]`; if $p > k - 1$ recurse left; else recurse right. This is

QuickSelect: $O(n)$ average.

Draw on Paper 1: Merge sort recursion tree

Draw the full recursion tree for merge sort on an array of 8 elements. Label each node with its subarray range. At each level, write the total comparisons in the merge step. Show why the total is $O(n \log n)$.

Submission. Save files as `lab11_ex*.java` and submit on the course site.